Auth & Authorization Service

1. Overview

The Auth & Authorization Service is a small, fast microservice that answers two questions for any application that calls it: **who is making this request**, and **are they allowed to do what they are asking**?

It checks the caller's identity using Auth0 tokens, then looks up fine-grained permission rules stored in a local SQLite database. Every decision is recorded, so there is always a trail showing who asked for what and why the answer was yes or no. The service can be used through a normal REST API or through an interactive terminal control panel, so no web front end is needed.

2. The Problem This Service Solves

In systems made of many services, access control often goes wrong for the same few reasons:

1. **Login and permissions get mixed together.** Many applications treat "who are you?" and "what can you do?" as one question and hardcode roles inside each service. This makes access rules rigid and hard to adapt.
2. **Permissions are hard to change.** Updating what a user can do usually means editing code, redeploying, or changing database structures in several places.
3. **Nested resources are awkward to describe.** Rules like "everything under wallets" versus "only this one wallet's transactions" are difficult to express with typical role systems.
4. **Conflicting rules cause confusion.** Without a clear way to decide between an allow rule and a deny rule, results become unpredictable and can open the door to privilege escalation.
5. **There is no central record of decisions.** Compliance requires knowing exactly what was decided, when, and for what reason.

3. The Solution

The service acts as a central decision-maker (a Policy Decision Point) and an authentication gateway. Its main features are:

- **Auth0 token verification.** It validates the caller's JWT (signed with RS256) using Auth0's public keys and reads the user's identity from the `sub` claim.
- **Dynamic policy rules.** Rules live in an embedded SQLite database (via `better-sqlite3`). Each rule ties together a user, an action, a resource, and an effect (`allow` or `deny`). Rules can be added, changed, or removed at any time without redeploying anything.
- **Smart wildcard matching.** Resource patterns can use `*` to stand for one path segment, and a rule on a collection also covers the items inside it. When several rules match, the most specific one wins, and an explicit `deny` beats an `allow` at that same level of specificity.
- **Permanent audit logs.** Each authorization check is saved with the user, method, path, decision, and the reason behind it.
- **Two ways to use it.** Use the REST API from other services, or use the terminal control panel for administration and testing.

4. How a Request Is Handled

When a client wants to know whether an action is allowed, it follows these steps:

1. **Request arrives.** The client sends `POST /authorize` with either an Auth0 token (in the body as `access_token` or in the `Authorization: Bearer` header) or a plain `user_id`, together with the HTTP `method` and the `path` being accessed.
2. **Identity is verified.** If a token is supplied, the service checks its signature against Auth0's published keys (`.well-known/jwks.json`) and confirms the issuer and audience. The `sub` claim becomes the user ID. If a `user_id` was sent directly, that is used instead.
3. **Method and path are cleaned up.** The HTTP method is mapped to an action: `GET` becomes `read`; `POST`, `PUT`, and `PATCH` become `write`; `DELETE` becomes `delete`. The path has any query string and leading or trailing slashes removed.
4. **Rules are fetched.** The service pulls from the database every rule that belongs to that user and matches that action.
5. **Rules are matched and scored.** Three kinds of matches are recognised:
    - an exact match, such as `transactions`
    - a wildcard match, such as `wallets/*` matching `wallets/wallet-789`
    - a collection match, such as `transactions` covering `transactions/txn-456`
6. **Conflicts are resolved.** The most specific matching rules are kept. If any of them is a `deny`, access is refused. If no rule matches at all, access is refused by default.
7. **The decision is logged.** The result, the time, and the reasoning are written to the `authorization_logs` table.
8. **A response is returned.** The client receives a structured JSON answer.

5. Architecture

The system is arranged in four layers:

Layer	What it contains
Clients	Front-end apps, other microservices, the terminal control panel, and tools such as Postman or automated tests.
Gateway and authentication	An Express 5 server on port 3000 with CORS and JSON parsing, plus the `jose` library that verifies RS256 tokens using Auth0's public keys.
Policy engine	The method-to-action mapper, path normaliser, wildcard matcher, and the specificity scorer that resolves conflicts.
Data storage	A SQLite file (`permissions.db`) holding the `user_permissions` and `authorization_logs` tables.

Requests flow from the clients into the gateway, through token verification, then into the policy engine. The engine reads rules from the database and writes each decision back as an audit record.

Main Components

Component	File	Responsibility
API gateway	src/app.ts	Express application: CORS, JSON handling, and routing.
Auth verifier	src/auth/validateToken.ts	Validates Auth0 RS256 tokens with jose, caching the remote keys and checking claims.
Policy service	src/services/permissionService.ts	The decision-maker: path normalisation, wildcard evaluation, specificity scoring, and conflict resolution.
Database layer	src/database/db.ts	Manages the SQLite database holding rules and audit logs.
Permission routes	src/routes/permissions.ts	REST endpoints to create, list, update, and delete rules.
Terminal CLI	scripts/cli.ts	Interactive menu and one-line commands for managing the whole system.

6. Technology Stack

Category	Technology	Purpose
Runtime and language	Node.js 20+ and TypeScript 7	Modern ES modules, strong typing, and good async performance.
Web framework	Express v5	Routing, middleware, and request handling.
Identity and security	Auth0 and jose v6	Issuing machine-to-machine tokens, verifying RS256 signatures, and validating keys.
Database	SQLite 3 with better-sqlite3	Fast, embedded, zero-setup storage with ACID guarantees.
Validation	Zod	Checks the shape and type of incoming data at runtime.
Dev tooling	tsx	Runs TypeScript directly for development, scripts, and the CLI.
Testing	Vitest and Supertest	Unit, pattern-matching, and end-

Category	Technology	Purpose
		to-end HTTP tests.

7. Getting Started

7.1 Install

Make sure Node.js 20 or newer is installed, then run:

```
cd auth-authorization-service
npm install
```

7.2 Configure the environment

Create a `.env` file in the project root:

```
AUTH0_DOMAIN=dev-gx56u528ljl5u6mo.us.auth0.com
AUTH0_AUDIENCE=https://auth-service.example.com
PORT=3000
```

7.3 Set up the database

This loads the starting permission rules into `permissions.db`:

```
npm run db:init
```

7.4 Run the server

```
npm run dev      # development mode with live reload
npm run build    # build the production bundle
npm start        # run the production server
```

7.5 Run the tests

```
npm test
```

This runs the unit tests, the wildcard tests, and the API integration tests.

8. Terminal Control Panel (CLI)

The control panel lets administrators manage and test policies without Postman or a browser.

8.1 Interactive mode

```
npm run cli
```

You can also use `npm run perm`. The menu offers:

Option	What it does
1	List all permissions in a table with colour badges.
2	Search and filter by user ID, action, resource, or effect.
3	Add a new permission, step by step.

Option	What it does
4	Edit an existing permission by ID.
5	Quickly flip a rule between ALLOW and DENY.
6	Delete a permission by ID.
7	Simulate an access check (user, method, path).
8	View recent audit logs with reasons and timestamps.
9	Clear the audit logs.
10	Fetch an Auth0 token and test it live.
0	Exit.

8.2 Single-command mode

```
npm run cli stats                          # dashboard and database statistics
npm run cli list                           # list all permissions
npm run cli list user123                   # filter by user or keyword
npm run cli add user999 read reports allow         # add: <user_id> <action> <resource> <effect>
npm run cli add user999 delete reports deny
npm run cli edit 5 read wallets/user-123 allow     # edit: <id> [action] [resource] [effect]
npm run cli toggle 5                       # flip ALLOW and DENY for rule 5
npm run cli delete 5                       # delete rule 5
npm run cli test user123 GET /transactions         # simulate: <user_id> <method> <path>
npm run cli test user123 DELETE /transactions
npm run cli logs 20                        # show the 20 latest audit entries
npm run cli clear-logs                     # clear audit logs
npm run token                              # get an Auth0 token and run a live test
```

9. Auth0 Credentials and Getting a Token

The service uses one pre-configured Auth0 machine-to-machine (M2M) application. Its settings are listed below, followed by three ways to fetch an access token.

9.1 Auth0 configuration

Parameter	Value	Description
Auth0 domain	`dev-gx56u528ljl5u6mo.us.auth0.com`	The Auth0 tenant that hosts the authorization server and signing keys.
JWKS URL	`https://dev-gx56u528ljl5u6mo.us.auth0.com/.well-known/jwks.json`	Public RS256 key set that `jose` uses to verify tokens.
API identifier (audience)	`https://auth-service.example.com`	The expected `aud` claim in every token.

Parameter	Value	Description
Client ID (api_key)	PIAvI4kXAiP5f2Twn3Qf0qufp0bPOHim	Client ID of the M2M application.
Client secret	qz56WcSudCrWJyPJSIYzuxl8MIt5fIpxJlpW_Z5x31Z2kUxPUzpFerVbmcVZjrR5	Kept in .env or a secrets manager, never written into documentation.
Subject (sub)	PIAvI4kXAiP5f2Twn3Qf0qufp0bPOHim@clients	The identity read from a verified token and used for policy checks.

> ***Security note:*** *The client secret from the source README is deliberately left out of this document. Because it appeared in plain text, rotate it in the Auth0 dashboard and store the new value only in .env or a secrets manager.*

9.2 Fetching an access token

Method 1: the built-in script (recommended). It requests an M2M token from Auth0, makes sure the client has a permission in the SQLite database, and then runs a test authorization check.

```
npm run token
```

Method 2: cURL (OAuth 2.0 client credentials).

```
curl --request POST \
  --url https://dev-gx56u528ljl5u6mo.us.auth0.com/oauth/token \
  --header 'content-type: application/json' \
  --data '{
    "client_id": "PIAvI4kXAiP5f2Twn3Qf0qufp0bPOHim",
    "client_secret": "qz56WcSudCrWJyPJSIYzuxl8MIt5fIpxJlpW_Z5x31Z2kUxPUzpFerVbmcVZjrR5",
    "audience": "https://auth-service.example.com",
    "grant_type": "client_credentials"
  }'
```

Method 3: PowerShell.

```
$body = @{
    client_id     = "PIAvI4kXAiP5f2Twn3Qf0qufp0bPOHim"
    client_secret = " qz56WcSudCrWJyPJSIYzuxl8MIt5fIpxJlpW_Z5x31Z2kUxPUzpFerVbmcVZjrR5"
    audience      = "https://auth-service.example.com"
    grant_type    = "client_credentials"
} | ConvertTo-Json

$response = Invoke-RestMethod -Method Post -Uri "https://dev-gx56u528ljl5u6mo.us.auth0.com/oauth/token" -
ContentType "application/json" -Body $body
$response.access_token
```

10. REST API Reference

#	Method and path	Purpose
1	POST /authorize	Evaluate whether a request is allowed.

#	Method and path	Purpose
2	`GET /authorize/logs`	Return the 50 most recent decisions.
3	`DELETE /authorize/logs`	Clear all audit logs.
4	`GET /permissions`	List permissions. Optional filter: `?user_id=user123`.
5	`POST /permissions`	Create a permission rule.
6	`PATCH /permissions/:id`	Update a permission rule.
7	`DELETE /permissions/:id`	Delete a permission rule.
8	`GET /health`	Health check; returns `{"status": "OK"}`.

10.1 Evaluate authorization

Send `Content-Type: application/json`. The `Authorization: Bearer <token>` header is optional if the token is in the body.

```
{
 "access_token": "eyJhbGciOiJSUzI1NiIs...",
 "method": "GET",
 "path": "/transactions"
}
```

Instead of a token, you may send `"user_id": "user123"` directly.

10.2 Create a permission rule

```
{
 "user_id": "user123",
 "action": "read",
 "resource": "wallets/*",
 "effect": "allow"
}
```

10.3 Update a permission rule

```
{
 "action": "delete",
 "resource": "wallets/wallet-999",
 "effect": "deny"
}
```

11. Example Outputs

11.1 Access allowed

```
{
 "decision": "ALLOW",
 "user_id": "user456",
```

```
  "reason": "User has read permission for wallets/wallet-789/transactions",
  "matched_permissions": [
    {
      "action": "read",
      "resource": "wallets/*/transactions",
      "effect": "allow"
    }
  ]
}
```

11.2 Access denied

```
{
  "decision": "DENY",
  "user_id": "user123",
  "reason": "A matching deny permission blocks delete transactions/txn-456",
  "matched_permissions": [
    {
      "action": "delete",
      "resource": "transactions/txn-456",
      "effect": "deny"
    }
  ]
}
```

11.3 Invalid or expired token (HTTP 401)

```
{
  "decision": "DENY",
  "error": "Unauthorized",
  "reason": "Invalid or expired access token",
  "details": "JWT verification failed: signature is invalid"
}
```

11.4 CLI simulator

```
> npm run cli test user123 GET /transactions

  Database: Connected (SQLite) | Engine: Active
  Statistics: Total Rules: 14 | Allows: 11 | Denies: 3 | Unique Users: 6

Authorization Decision:
  User:     user123
  Method:   GET
  Path:     /transactions
  Action:   read
  Decision: ALLOW
  Reason:   User has read permission for transactions

Matched Rules (1):
  [ID 1] user123 - read transactions -> ALLOW
```

11.5 Audit log response (`GET /authorize/logs`)

```
{
  "count": 1,
  "logs": [
```

```
    {
      "id": 42,
      "user_id": "user123",
      "method": "GET",
      "path": "/transactions",
      "decision": "ALLOW",
      "reason": "User has read permission for transactions",
      "created_at": "2026-10-16 11:25:35"
    }
  ]
}
```

12. Summary

This service separates identity from permissions, keeps rules flexible and easy to change, resolves conflicts in a predictable way, and records every decision for auditing. Whether it is called from another service or managed from the terminal, the behaviour is the same: verify the caller, match the most specific rule, deny by default, and log the result.